

K1NDL1NG

A parser and a normaliser for the kernel of rholang
served by CAMPF1R3, as composable crates

Specification v0.2

F1R3FLY.io

17 September 2026

Abstract

CAMPF1R3 is a store: it holds data and continuations at channels and fires a matching step. It deliberately knows nothing about processes, and lists the parser and the normaliser as non-goals. RHOBOTS is an immersive editor for pure rho on Apple Vision Pro, and needs the same terms in the same shapes, on a device where a C-backed parser is unwelcome. This document specifies the two components that fill those holes, as *separate, independently usable* crates: `kindling-parse`, which takes text to a typed abstract syntax tree, and `kindling-norm`, which takes that tree to a representation with de Bruijn indices and ordered parallel components, from which name equality is a byte comparison. Both are built over a shared `kindling-ast`, neither depends on the other's consumers, and neither reduces anything. Composed with CAMPF1R3 and an interpreter, they make an executive (Section 3.3). The specification fixes the nomenclature (Section 1.3) and the grammar (Section 2); the tree (Section 4), the parser (Section 5) and the normaliser (Section 6) crate by crate; the error model (Section 8), the editor services RHOBOTS needs (Section 9), and the portability, determinism and performance requirements (Sections 10–11). Key words `MUST`, `MUST NOT`, `SHOULD`, `MAY` are used in the sense of RFC 2119.

Contents

1	Purpose and scope	2
1.1	The gap	2
1.2	What exists today	3
1.3	Nomenclature	3
1.4	Goals	4
1.5	Non-goals	4
1.6	Sources	5
2	The language	5
2.1	Three levels	5
2.2	Lexical structure	5
2.3	Grammar	6
2.4	Out of scope, explicitly	6
2.5	Congruence	7
3	Architecture	7
3.1	The crates	7
3.2	The pipeline, as crate boundaries	9
3.3	Composing an executive	9

4	The tree	10
4.1	Types	10
4.2	Well-formedness	11
4.3	Construction	11
5	The parser	11
6	The normaliser	12
6.1	Binders	12
6.2	Unforgeable names	13
6.3	The normal form and its encoding	13
6.4	Correctness	14
6.5	Hashing	14
7	Instantiating CAMPF1R3	15
7.1	The WebAssembly interface	15
8	Errors and recovery	16
9	Editor services	16
9.1	Stable identity	16
9.2	Subterm reparse and splice	17
9.3	Printing	17
9.4	Scene queries	17
10	Performance	18
10.1	Complexity	18
10.2	Targets	18
11	Portability, determinism and dependencies	19
12	Validation	20
13	Open decisions	20
14	Delivery	21

1 Purpose and scope

1.1 The gap

The CAMPF1R3 specification (v0.1) covers the store and the matching step, and is explicit that “the parser and the normaliser are non-goals”. Its WebAssembly build is instantiated at $C = A = K = \text{Bytes}$ with a placeholder `ExactBytes` matcher, precisely so that the store can be exercised in a browser *before* a parser is available. That placeholder is the hole this document fills.

The same hole is open on the other side. RHOBOTS renders pure rho as robots exchanging payloads, with the z axis carrying continuations. It needs to turn typed text into terms, terms back into text, and terms into a scene graph with stable identity across edits; and it needs all of that on a headset, in a build where linking two and a half megabytes of generated C is a poor use of the budget.

1.2 What exists today

Component	State
<code>rholang-parser</code> (<code>rholang-rs</code> , rev 02cef80)	Tree-sitter front end for the whole of <code>rholang</code> , pinned as a git dependency by <code>f1r3node-rust</code> . The generated <code>src/parser.c</code> is 2.75 MB; the crate carries an arena-allocated AST (<code>ast.rs</code> , 913 lines) and a traversal layer (1851 lines).
<code>rholang-parser</code> on <code>wasm32</code>	The tree-sitter backend is compiled out, and <code>parser_wasm.rs</code> substitutes a stub whose <code>parse</code> returns <code>Validated::Good(vec![])</code> . Every program is silently the empty process.
<code>rholang/.../compiler</code> (<code>f1r3node-rust</code>)	<code>Compiler::source_to_adt</code> plus a normaliser of 23 per-construct visitors, producing <code>models::rhoapi::Par</code> . Depends on <code>models</code> , <code>prost</code> and the rest of the workspace.
<code>models/.../sorter</code>	Score-tree canonical ordering over <code>Par</code> , 15 sort matchers, the basis of the Blake2b-256 content hashes <code>rspace++</code> already computes.
<code>rho-pure-eval</code>	Deterministic evaluator for a subset of <code>Expr</code> , used for <code>where</code> guards and match-case guards. 1042 lines.

Remark 1.1 (Why not simply reuse the tree-sitter front end). Three reasons, in order of weight. First, it does not work on the target: the `wasm32` path is a stub that returns an empty AST rather than an error, so a RHOBOTS build or a browser CAMPF1R3 demo would run, and would be wrong. Second, it parses the whole language, which is the wrong size for a store whose kernel is six constructors; grammar surface that nothing downstream can execute is surface that must still be diagnosed, printed and tested. Third, the canonical form and the content hash — the things CAMPF1R3 actually requires of its caller — live in `models`, behind `prost`, `bincode` and the workspace, none of which CAMPF1R3 is permitted to depend on.

Remark 1.2 (What is kept). `K1NDL1NG` is not a rejection of that work. The grammar of `rholang-tree-sitter` is normative for surface syntax: where `K1NDL1NG` accepts a form, it MUST accept it with the same spelling and the same meaning, so that a kernel program is a `rholang` program. The AST shape (`Proc`, `Name`, `Bind`, `Source`, `SendType`) is followed where the kernel overlaps it, so that a later merge or replacement is mechanical rather than a redesign. The tree-sitter grammar and this crate are cross-checked by differential testing (Section 12).

1.3 Nomenclature

Three words are used precisely throughout, and nowhere interchangeably.

Term	Meaning in this document
<i>Parsing</i>	Taking a textual representation to a typed abstract syntax tree. Nothing more: no binder analysis, no reordering, no equality. The output is a faithful tree over the source, with spans.
<i>Normalisation</i>	Taking that tree to a representation that uses de Bruijn indices and orders the components of a parallel composition, together with the encoding and content hash that follow from it. This is what makes name equality cheap, and it is the whole reason the stage exists.
<i>Reduction</i>	Execution: firing communications, including running a term to a normal form in the operational sense. No crate in this document performs any reduction.

Remark 1.3 (Two senses of “normal form”). CAMPF1R3 uses *normal form* for a store state in which no match remains. That is a property of a store, reached by reduction. The *normal form* of this document is a property of a term, reached by normalisation, and no term is ever reduced to get it. Where confusion is possible, this document says *normal form of a term* or *quiescent store*.

Decision 1.4 (One document, two crates). Parsing and normalisation are separate components and are specified as separate crates. A consumer that only needs a tree — an editor, a formatter, a syntax highlighter, a gesture interface building terms by hand — MUST NOT be made to link a normaliser, and a consumer that only needs terms — an interpreter, a matcher, a store client fed by the wire rather than by text — MUST NOT be made to link a lexer. Section 3 gives the boundary; the remainder of the document is organised crate by crate.

1.4 Goals

1. **Separable.** Each crate is usable, testable and versionable on its own, and the dependency graph is a line, not a cycle (Requirement 3.1).
2. **Composable.** Parser, normaliser, CAMPF1R3 and an interpreter assemble into an executive with no glue beyond type instantiation, on a headset, in a browser or on a node (Section 3.3).
3. **Small.** One kernel, six process constructors and two name constructors at level K0; each crate in the low thousands of lines, with a `wasm32` code-size budget three orders of magnitude under the generated parser it replaces (Section 10).
4. **Tight.** Parse, normalise and hash in one pass each, over an arena, with no intermediate `String` and no regular expression engine.
5. **Portable.** `wasm32-unknown-unknown` is a MUST target, and a first-class one: no stubs, no silent degradation, no C, no `getrandom`, no clock.
6. **Two-way.** Trees print back to source, and normal forms print back to trees. RHOBOTS builds terms by gesture and must be able to show and export them as text.

1.5 Non-goals

Reduction in any form, matching, scheduling, cost accounting, storage, consensus, the name registry, unforgeable-name derivation compatible with a particular deploy scheme, the `Expr` layer of rholang beyond the guard subset, and language-definition (DDL) syntax. Each is named where it touches an interface, and each belongs to another crate. A term here is data.

Decision 1.5 (Substitution belongs to the normaliser). Capture-avoiding substitution and de Bruijn shifting are *included* in `kindling-norm`, though a substitution is not a normalisation. They are operations on the de Bruijn representation, they are small and total given that representation, and an interpreter that had to reimplement them against someone else’s indices would be reimplementing the normaliser’s invariants. Applying a substitution is not reduction: the interpreter decides what to substitute and when.

1.6 Sources

- Store interface and its obligations on the caller: the CAMPF1R3 specification v0.1, especially the hash discipline, the dependency allowlist and the determinism profiles.
- Surface syntax: `rholang-tree-sitter/grammar.js` and `rholang-parser/src/ast.rs`, `rholang-rs` rev 02cef80.
- Canonical order and content hashing: `models/src/rust/rholang/sorter`, `f1r3node-rust`, branch `feature/cost-accounted-rho`.
- Guard sublanguage: `rho-pure-eval`, same branch.
- Consumer requirements: *Rhobots in Space* v0.6.
- Calculus: the rho calculus as presented in the f1r3lang papers in `publications`; reflection, and the laws $@*x = x$ and $*@P \equiv P$.

2 The language

2.1 Three levels

The kernel is stratified, so that a consumer can state exactly what it accepts and a diagnostic can say exactly why a form was rejected.

Level	Adds	Consumer
K0 <i>pure rho</i>	Nil, par, monadic send, monadic receive on a name variable, unquote, quote	RHOBOTS v1
K1 <i>store kernel</i>	polyadic send and receive, joins (<code>&</code>), persistent receive (<code><=</code>), peek (<code><-</code>), persistent send (<code>!!</code>), name patterns with wildcards and free variables, <code>new</code> , sequential receipts (<code>;</code>) as sugar	CAMPF1R3, node adapter
K2 <i>guards</i>	<code>where</code> clauses on receipts, over the <code>rho-pure-eval</code> expression subset	deferred, feature-gated

Requirement 2.1 (Level is a parser parameter). The parser MUST take the accepted level as an explicit parameter, MUST reject forms above that level with a diagnostic naming the level that would accept them, and MUST NOT silently accept them. K0 and K1 MUST be implemented in v1. K2 MUST be behind the `guards` cargo feature, off by default, and MAY be deferred to v1.1.

Decision 2.2 (Why K0 is not merely “K1 with one binder”). RHOBOTS renders a robot per constructor. Arity, joins, persistence and peek each need a distinct physical form, and the spec reserves but does not fix them. Making K0 a real level, checkable by the parser, lets the editor refuse to load a term it cannot draw, rather than drawing it wrongly.

2.2 Lexical structure

Class	Definition
Identifier	<code>[a-zA-Z_][a-zA-Z0-9_']*</code> , not a reserved word. ASCII only in v1; the lexer MUST reject non-ASCII identifier bytes with a diagnostic rather than accept them silently.
Wildcard	<code>_</code> standing alone.
Punctuation	<code>() { } @ * ! !! & ; , <- <= <- new in</code>
Comment	<code>//</code> to end of line; <code>/* ... */</code> , not nested.
Whitespace	space, tab, carriage return, line feed. No layout rule.
Reserved	Nil new in for where contract match select let if else bundle bundle0 bundle+ bundle- true false not and or matches

Requirement 2.3 (Reserved words are reserved at every level). Words reserved for constructs above the accepted level **MUST** be lexed as keywords and **MUST** produce a “not in this kernel” diagnostic, never an identifier. This is what keeps a kernel program a rholang program, and keeps a future level from silently changing the meaning of an old term.

2.3 Grammar

Nonterminals are italic; terminals are typeset in **typewriter**. The suffix ^{K1} marks a production available only at K1 and above, and ^{K2} at K2.

<i>Proc</i>	::=	Nil	the inert process
		<i>Proc</i> <i>Proc</i>	parallel composition
		Name <i>Bang</i> (<i>Proc</i> [*])	send
		for (<i>Receipts</i>) { <i>Proc</i> }	receive
		* <i>Name</i>	unquote
		new <i>Id</i> ⁺ , in { <i>Proc</i> }	name creation ^{K1}
		(<i>Proc</i>)	
<i>Name</i>	::=	<i>Id</i> ₋ ^{K1} @ <i>Proc</i>	
<i>Bang</i>	::=	! !! ^{K1}	
<i>Receipts</i>	::=	<i>Receipt</i> (; <i>Receipt</i>) [*] K1	
<i>Receipt</i>	::=	<i>Bind</i> (& <i>Bind</i>) [*] K1 (where <i>Guard</i>) ^{?K2}	
<i>Bind</i>	::=	<i>NamePat</i> [*] , <- <i>Name</i>	linear
		<i>NamePat</i> [*] , <= <i>Name</i>	persistent ^{K1}
		<i>NamePat</i> [*] , «- <i>Name</i>	peek ^{K1}
<i>NamePat</i>	::=	<i>Id</i> ₋ ^{K1} @ <i>ProcPat</i> ^{K1}	
<i>ProcPat</i>	::=	a <i>Proc</i> with free identifiers and ₋ ^{K1}	

X^* is a possibly empty comma-separated list, X^+ a non-empty one. At K0, a *Bind* has exactly one *NamePat*, which is an *Id*; a send has exactly one argument; *Receipts* is a single *Bind* with <-.

Precedence	Form	Associativity	Note
lowest		left, but see below	flattened, then sorted
	for , new	—	body is brace-delimited
	!, !!	—	argument list is paren-delimited
highest	*, @	prefix, right	*@P and @*x are legal

Remark 2.4 (Associativity of | is not observable). The parser **MAY** read | left-associated, but the tree **MUST** record parallel composition as a flat sequence of components in source order (Requirement 4.2); ordering that sequence is the normaliser’s business, not the parser’s. Parsing | into a right-leaning spine of **Par{left,right}** nodes, as the tree-sitter AST does, forces every consumer to re-flatten.

2.4 Out of scope, explicitly

Ground literals of every sort, collections, methods, **contract**, **match**, **select**, **let**, **if/else**, bundles, **!?** and **?!** sources, method-call source sugar, **VarRef** (**=x**), logical connectives in patterns (**/**, **\/**, **~**), **matches**, and simple types. All are reserved words or reserved punctuation; all produce a level diagnostic.

Obligation 2.5 (Connectives in patterns). CAMPF1R3 treats patterns as opaque and delegates to a caller-supplied matcher, so nothing in the store forbids connectives. They are excluded from v1 because the kernel’s matcher (a separate deliverable) is specified for spatial patterns with free variables and wildcards only. If the node adapter is to serve `f1r3node` terms in general, connectives are the first extension, and the grammar above reserves their tokens for that purpose.

2.5 Congruence

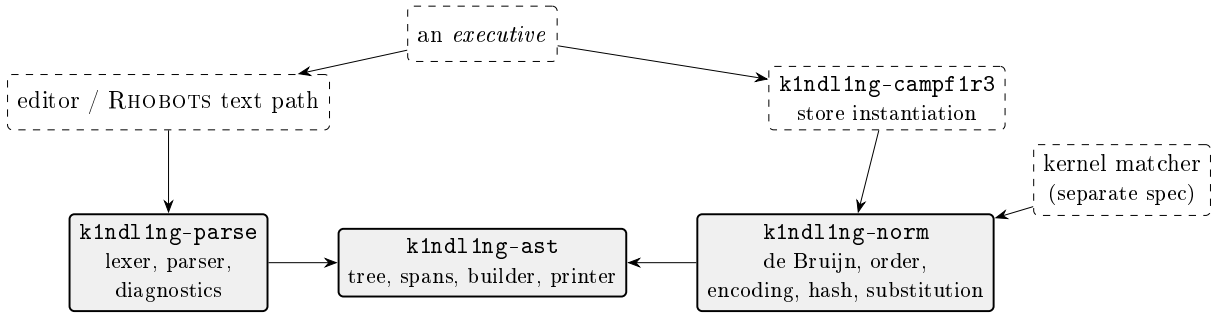
Definition 2.6 (Structural congruence). $\equiv$ is the least congruence on $Proc$ such that $(Proc, |, Nil)$ is a commutative monoid, α -equivalent terms are identified, and $*@P \equiv P$. Name equivalence $\equiv_N$ is the least congruence on $Name$ such that $@P \equiv_N @Q$ whenever $P \equiv Q$, and $@*x \equiv_N x$.

Requirement 2.7 (Congruence is decided, not approximated). For every pair of kernel terms P, Q , $\llbracket P \rrbracket = \llbracket Q \rrbracket$ if and only if $P \equiv Q$, where $\llbracket - \rrbracket$ is the normal-form encoding of Section 6. Equality of terms, equality of encodings and equality of content hashes (modulo collision) MUST agree.

Remark 2.8. Requirement 2.7 is what CAMPF1R3 is really asking for when it says the caller supplies content hashes and that “equal values MUST have equal hashes”. Two channels that are congruent but encode differently would split one channel into two, and messages would never meet. This is the single most load-bearing property in this document, and Section 6 proves it for the kernel.

3 Architecture

3.1 The crates



Crate	Owns	Depends on
<code>kindling-ast</code>	The typed tree: node types, arena, spans, trivia, node identity, traversal, the construction API, and printing a tree back to text. This is the vocabulary the other two share.	<code>core</code> , <code>alloc</code>
<code>kindling-parse</code>	Lexer, parser, error model, recovery, fragment parsing and splice. Text in, tree out.	<code>kindling-ast</code>
<code>kindling-norm</code>	Binder resolution, de Bruijn assignment, ordering of parallel components, the normal form, its encoding, content hashing, substitution and shifting, and queries over normal forms.	<code>kindling-ast</code>
<code>kindling-wasm</code>	<code>wasm-bindgen</code> bindings, one module per crate behind cargo features, so a browser build can take the parser without the normaliser or the reverse.	both, optional
<code>kindling-campfir3</code>	Instantiation of <code>CAMPFIR3</code> 's <code>Channel</code> , <code>Datum</code> , <code>Pattern</code> and <code>Cont</code> at normal-form types (Section 7). A glue crate, deliberately trivial.	<code>kindling-norm</code> , <code>campfir3-core</code>
<code>kindling-par</code>	Translation to and from <code>models::rhoapi::Par</code> , and the minter reproducing the node's unforgeable-name scheme. Lives in the <code>f1r3node-rust</code> workspace, as <code>campfir3-ospace</code> does.	<code>kindling-norm</code> , <code>models</code>

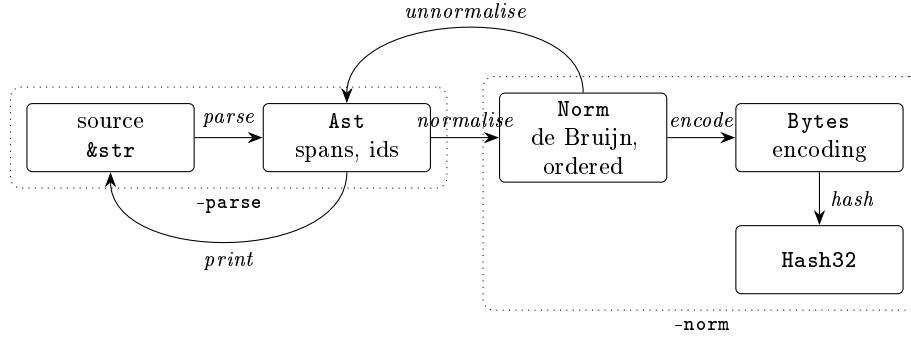
Requirement 3.1 (The graph is acyclic and shallow). `kindling-parse` and `kindling-norm` MUST NOT depend on each other, and neither MUST depend on `campfir3`, on any interpreter, or on any crate of the `f1r3node-rust` or `rholang-rs` workspaces. Composition happens in a glue crate or in the executive, never by one component reaching into another. `campfir3` MUST NOT depend on any crate in this document.

Requirement 3.2 (Normalisation does not require parsing). `kindling-norm` MUST accept any tree satisfying the well-formedness conditions of Section 4, however it was built: parsed, constructed through the builder by a gesture interface, or decoded from a normal form. A consumer that never handles text MUST NOT link a lexer.

Requirement 3.3 (Parsing does not require normalisation). `kindling-parse` MUST produce a complete, printable, navigable tree without assigning a de Bruijn index or ordering anything. An editor that never asks about equality MUST NOT link a normaliser. In particular, the parser MUST NOT reorder the components of a parallel composition, and MUST NOT reject a term for a binding error; binding errors are reported by the normaliser (Section 6.1).

Remark 3.4 (Where the old draft put the boundary wrongly). Version 0.1 of this specification described one crate with five stages, and called the second half “canonicalisation”. Two things were wrong with that. The work is normalisation in the ordinary sense — de Bruijn indices and ordered pars — and calling it something else hid the fact that a second component had appeared inside the first. And the single crate forced every consumer to take both halves, which is exactly what a headset renderer and a wire-fed interpreter each do not want.

3.2 The pipeline, as crate boundaries



`unnormalise` takes a normal form back to a tree, with generated identifiers in place of indices, so that a term arriving from the store or the wire can be printed and shown. It is in `kindling-norm`, since only the normaliser knows what the indices meant.

3.3 Composing an executive

An *executive* is what you get when these crates are put together with a store, a matcher and an interpreter. Nothing in this document is an executive; this subsection exists to show that the boundaries chosen above are the ones an executive actually wants.

Role	Supplied by
Text to tree	<code>kindling-parse</code>
Tree to normal form, hashes, substitution	<code>kindling-norm</code>
Data and continuations at channels, the cut	<code>campf1r3-core</code> , instantiated through <code>kindling-campf1r3</code>
Spatial match of a pattern against a datum	the kernel matcher: a <code>campf1r3 Match</code> implementation over normal forms. Not specified here; its interface is fixed by <code>CAMPF1R3</code> and its input invariants by Section 6
Choosing what runs, firing it, and the substitution that follows	the interpreter
Wiring, policy, scheduling, presentation	the executive

```

// The whole of an executive's deploy path, in outline.
let ast = kindling_parse::parse(src, &opts);           // or built by gesture
let term = kindling_norm::normalise(&ast.tree, &opts?); // de Bruijn + ordered
for p in term.top_level() {                           // rho reduces at top par
    match p.shape() {
        Shape::Send { chan, args, persistent } =>
            space.produce(chan.clone(), args.to_vec(), persistent)?,
        Shape::Receive { binds, body } =>
            space.consume(join_of(binds), pats_of(binds), cont_of(body), ...)?,
        Shape::New { .. } | Shape::Nil | Shape::Eval { .. } =>
            interpreter.handle(p)?,
    };
}
// A comm result carries a continuation and the data it matched.
// The interpreter substitutes and re-enters the loop; the store decided the race.

```

Remark 3.5 (Three executives, one set of crates). The headset executive links `-parse`, `-ast`, `-norm`, the matcher, `campfir3-core` and a local interpreter into one `aarch64-apple-visionos` binary, with no network in the loop. The browser demo links the same set for `wasm32`, replacing `CAMPF1R3`'s `ExactBytes` placeholder with real terms. The node keeps its own reducer and storage and takes `kindling-par` only, to read kernel terms into `Par`. The three differ in what they wire, not in what they contain.

Requirement 3.6 (No executive in these crates). No crate in this document **MUST** contain a run loop, a scheduler, a policy for choosing among competing communications, or a call into a store. The sketch above is documentation, and **MAY** ship as an example under `examples/`, where it **MUST** be feature-gated and **MUST NOT** be a dependency of any library target.

4 The tree

4.1 Types

`kindling-ast` defines the tree and nothing else. It is deliberately independent of how the tree arose.

```
pub struct Ast<'src> {
    procs: Vec<ProcNode<'src>>, // arena; index is ProcId
    names: Vec<NameNode<'src>>,
    spans: Vec<Span>,           // parallel to procs/names
    root: ProcId,
    trivia: Vec<Trivia>,        // comments and layout, for faithful printing
}

#[derive(Clone, Copy, PartialEq, Eq)] pub struct ProcId(u32);
#[derive(Clone, Copy, PartialEq, Eq)] pub struct NameId(u32);
#[derive(Clone, Copy)] pub struct Span { pub lo: u32, pub hi: u32 } // byte offsets

pub enum ProcNode<'src> {
    Nil,
    Par(ProcSlice), // flat, in source order
    Send { chan: NameId, persistent: bool, args: ProcSlice },
    Receive { receipt: ReceiptId, body: ProcId },
    Eval { name: NameId }, // *x
    New { names: IdSlice<'src>, body: ProcId },
    Var { id: &'src str }, // process variable
    Wild, // -
    Error { diag: DiagId },
}

pub enum NameNode<'src> { Var { id: &'src str }, Quote(ProcId), Wild }

pub struct Receipt { pub binds: BindSlice, pub guard: Option<ProcId> }
pub struct Bind { pub kind: BindKind, pub pats: NameSlice, pub chan: NameId }
pub enum BindKind { Linear, Persistent, Peek }
```

Remark 4.1 (Identifiers survive, indices do not exist yet). `New` carries the declared identifiers, not a count; `Var` carries the identifier text. A tree is about the program as written. Everything positional appears one stage later, which is what lets the same tree be printed back byte for byte.

Requirement 4.2 (Flat par). `Par` **MUST** hold a flat sequence of two or more components in source order. The parser **MAY** read `|` left-associated, but **MUST NOT** record a spine of binary nodes, and **MUST NOT** sort. Ordering is the normaliser's business; flatness is a property of the syntax, since `|` is associative and the tree shape carries no information the source had.

Requirement 4.3 (Portable arithmetic). Every index, offset, length and identifier in a public type **MUST** be a fixed-width type of `u32` or narrower, never `usize`, so that 32-bit WebAssembly and 64-bit native builds agree bit for bit. A source longer than $2^{32} - 1$ bytes **MUST** be rejected by the parser with a diagnostic.

Requirement 4.4 (Arena, not pointers). The tree **MUST** be stored in flat vectors indexed by `u32`. It follows that `Ast` is `Send`, cheap to clone in bulk, serialisable without pointer fixups, and free of `Drop` recursion; the last of these is what stops a deep term from overflowing the stack when it is released.

Requirement 4.5 (No recursion anywhere). Every traversal in every crate — parsing, printing, normalisation, encoding, substitution and `Drop` — **MUST** be iterative, driven by an explicit work stack on the heap. No function **MUST** recurse on term structure. The browser main thread's stack is small, `stacker` is unavailable on `wasm32`, and a 1 MB stack is reached by a few thousand nested quotes — which a gesture interface can produce by accident. A configurable `max_depth` (default 2^{16}) **MUST** additionally bound nesting and **MUST** report a diagnostic rather than abort.

4.2 Well-formedness

Definition 4.6 (Well-formed tree). A tree is well-formed when every `ProcId` and `NameId` is in range, the root is reachable, every `Par` has at least two components, no `Error` node occurs, and every `Span` lies within the source it was built from, or is the null span for a constructed tree.

Requirement 4.7 (Well-formedness is checkable and checked). `kindling-ast` **MUST** provide `Ast::check() -> Result<(), WfError>` in $O(n)$. `kindling-norm` **MUST** either call it or document that it is the caller's obligation, and **MUST NOT** have undefined behaviour on an ill-formed tree; the worst permitted outcome is a `WfError`. This is the contract that makes Requirement 3.2 safe for trees built by a gesture interface rather than by the parser.

4.3 Construction

```
pub struct Builder<'a> { /* arena */ }
impl<'a> Builder<'a> {
    pub fn nil(&mut self) -> ProcId;
    pub fn par(&mut self, xs: &[ProcId]) -> ProcId;
    pub fn send(&mut self, chan: NameId, persistent: bool, args: &[ProcId]) -> ProcId;
    pub fn receive(&mut self, binds: &[Bind], body: ProcId) -> ProcId;
    pub fn eval(&mut self, n: NameId) -> ProcId;
    pub fn new_names(&mut self, ids: &[&'a str], body: ProcId) -> ProcId;
    pub fn quote(&mut self, p: ProcId) -> NameId;
    pub fn name_var(&mut self, id: &'a str) -> NameId;
    pub fn finish(self, root: ProcId) -> Ast<'a>;
}
```

The builder is in `kindling-ast`, not in the parser, precisely so that RHOBOTS can build a term from gestures and normalise it without a lexer in the binary. A builder-produced tree **MUST** be well-formed by construction.

5 The parser

`kindling-parse` does one thing: text to a typed tree.

```
pub struct Options { pub level: Level, pub max_depth: u32, pub desugar_seq: bool }
pub enum Level { K0, K1, K2 }

pub fn parse<'src>(src: &'src str, opts: &Options) -> Parsed<'src>;
pub struct Parsed<'src> { pub tree: Ast<'src>, pub diags: Vec<Diag> }
```

```

/// Parse one process, for splicing into an existing tree.
pub fn parse_fragment<'src>(src: &'src str, opts: &Options) -> Parsed<'src>;

```

Stage	What it does and does not do
Lexer	One left-to-right pass, no backtracking, no regular expressions. Tokens carry a kind and a byte range and nothing else; text is recovered by slicing the source. Trivia is recorded in a side table, not in the token stream.
Parser	Explicit-stack descent (Requirement 4.5) producing Ast : arena nodes, u32 indices, spans, node identifiers. Identifiers are &str borrowed from the source. No binder analysis, no ordering, no equality, no desugaring beyond sequential receipts.

Requirement 5.1 (The parser has no opinion about binding). The parser **MUST** accept a syntactically valid term whose variables are unbound, duplicated in a pattern, or shadowed. Those are diagnostics of Section 6.1, produced by the normaliser. An editor showing a half-written term needs a tree for it; refusing to build one is the behaviour that makes structure editing impossible.

Decision 5.2 (Sequential receipts are the one desugaring). `for(a <- x; b <- y){P}` is parsed as nested receives when `desugar_seq` is set, and as a single receive node with two receipts when it is not. Default is set. This is the only sugar in the kernel, and the flag exists so that a formatter can round-trip the source form while an executive sees only the nested form. If the flag proves a nuisance, the honest alternative is to drop `;` from the kernel entirely.

Requirement 5.3 (Level is a parser parameter). The parser **MUST** take the accepted level (Section 2) as an explicit parameter, **MUST** reject forms above that level with a diagnostic naming the level that would accept them, and **MUST NOT** silently accept them. K0 and K1 **MUST** be implemented in v1; K2 **MUST** be behind the `guards` cargo feature, off by default.

6 The normaliser

`kindling-norm` takes a well-formed tree to a normal form: de Bruijn indices for bound occurrences, de Bruijn levels for free ones, ordered parallel components, and the encoding and content hash that follow. Name equality then costs a 32-byte comparison, which is the point of the exercise.

```

pub fn normalise(tree: &Ast<'_>, opts: &Options) -> Result<Norm, Vec<Diag>>;
pub fn unnormalise(t: &Norm) -> Ast<'static>;           // for printing and display

impl Norm {
  pub fn encode(&self) -> &[u8];                        // memoised
  pub fn hash<D: Digest32>(&self) -> Hash32;           // memoised
  pub fn decode(bytes: &[u8]) -> Result<Norm, DecodeError>;
  pub fn substitute(&self, env: &Subst) -> Norm;       // Decision 1.7
  pub fn shift(&self, by: i32, cutoff: u32) -> Norm;
  pub fn top_level(&self) -> &[Norm];                  // the par components
  pub fn free_names(&self) -> &[FreeName];
  pub fn size(&self) -> u32;
  pub fn depth(&self) -> u32;
}

```

6.1 Binders

Rho has two binding forms and one derived one. `for` binds names in its patterns; `new` binds names in its body; a quoted pattern `@{...x ...}` binds the process variable `x` at the position of the enclosing name pattern.

Definition 6.1 (Sorts). An identifier occurrence resolves to exactly one of: `NameBound(i)` — a bound name, de Bruijn index i ; `ProcBound(i)` — a process variable bound by a quoted pattern; `FreeName(1)` / `FreeProc(1)` — free, carrying the de Bruijn level of its first occurrence.

Requirement 6.2 (Static checks belong here). The normaliser **MUST** reject, with a diagnostic and the span carried by the tree: (i) a free variable occurring twice in one pattern; (ii) a wildcard where a bound occurrence is required (`*_`); (iii) a `new`-bound name used as a pattern variable; (iv) a term with free names, when the caller asked for a closed term. It **MUST NOT** reject free names otherwise.

Decision 6.3 (Free names are reported, not an error). `Norm` carries `free_names` in first-occurrence order, with the identifier and span from the tree. `RHOBOTS` synthesises its canonical outer comprehension on `stdin` from exactly this list; the node adapter uses it to decide whether a term is deployable. The normaliser takes no view. This differs deliberately from `f1r3node`’s `source_to_adt`, which errors when `free_map.count() > 0`, and the difference **MUST** be documented where the two meet.

6.2 Unforgeable names

`new` introduces names that quote nothing. `Norm` represents them as `Name::Unforgeable`, carrying 32 bytes minted through a trait rather than by a fixed scheme:

```
pub trait NameMinter { fn mint(&mut self, path: &BinderPath) -> [u8; 32]; }
pub struct Sequential; // pure function of binder position; the default
```

Decision 6.4 (Minting is the caller’s policy). `Sequential` is a pure function of the binder’s position in the normal form, so a `K1NDL1NG`-only pipeline is deterministic and reproducible. It is *not* compatible with `f1r3node`’s deploy-seeded derivation and **MUST NOT** be presented as such; `k1ndl1ng-par` supplies a minter that reproduces the node’s scheme, so compatibility is a property of the adapter, tested differentially, and not a constraint on the core.

6.3 The normal form and its encoding

Definition 6.5 (Encoding). $\llbracket - \rrbracket$ maps a normal form to a byte string. Each constructor is a tag byte followed by its fields; lengths are LEB128-encoded `u32`; identifiers never appear, only de Bruijn indices and levels.

Tag	Constructor	Payload	Ordered
0x00	<code>Nil</code>	—	—
0x01	<code>Par</code>	n , then $\llbracket P_1 \rrbracket \cdots \llbracket P_n \rrbracket$	yes, by $\llbracket P_i \rrbracket$
0x02	<code>Send</code>	$\llbracket x \rrbracket$, persistence bit, n , $\llbracket Q_1 \rrbracket \cdots \llbracket Q_n \rrbracket$	no (arity is ordered)
0x03	<code>Receive</code>	m , $\llbracket b_1 \rrbracket \cdots \llbracket b_m \rrbracket$, $\llbracket P \rrbracket$	yes, by $\llbracket b_j \rrbracket$
0x04	<code>New</code>	k , $\llbracket P \rrbracket$	—
0x05	<code>Eval</code>	$\llbracket x \rrbracket$ where x is a variable	—
0x06	<code>BoundVar</code>	index	—
0x07	<code>FreeVar</code>	level	—
0x08	<code>Wild</code>	—	—
0x10	<code>Quote</code>	$\llbracket P \rrbracket$	—
0x11	<code>BoundName</code>	index	—
0x12	<code>FreeName</code>	level	—
0x13	<code>Unforgeable</code>	32 bytes	—
0x20	<code>Bind</code>	kind byte, $\llbracket \text{chan} \rrbracket$, n , patterns	no

Requirement 6.6 (The order key is the encoding itself). Parallel components and the binds of one receipt **MUST** be ordered by bitwise lexicographic comparison of their own encodings, ascending. No separate score tree is computed, and no ordering **MUST** depend on source order, hash values, pointer values or iteration order of a hash container.

Remark 6.7 (Why not the node’s score tree). **models**’s sorter builds a score tree per term, sorts by it, and serialises separately, and in **f1r3node** the sort is a pass distinct from normalisation, invoked at substitution and storage time. For the kernel the encoding *is* a score tree, flattened: tags are assigned in the order the sorter’s **Score** constants assign them, so bitwise comparison agrees with score comparison on this fragment, and one pass does the work of two. Agreement with **models** is an obligation, not an assumption (Obligation 6.12).

Requirement 6.8 (Bottom-up, memoised). Normalisation **MUST** compute each subterm’s encoding once, bottom-up, and **MUST** reuse it as the order key of the parent, so that ordering a par of n components of size s costs $O(n \log n \cdot s)$ comparisons of already-built byte strings and not $O(n \log n)$ re-encodings.

6.4 Correctness

Proposition 6.9 (Normal form decides congruence). *For kernel terms P, Q with the same free-name environment, $\llbracket P \rrbracket = \llbracket Q \rrbracket$ iff $P \equiv Q$, with $\equiv$ as in Definition 2.6.*

Proof sketch. ($\Leftarrow$) By induction on the derivation of $\equiv$. Commutativity and associativity of $|$ are absorbed by flatness (Requirement 4.2) and by Requirement 6.6, since a total order on encodings gives one representative per multiset. **Nil** as unit is absorbed by dropping **Nil** components during normalisation, except for the empty par, which encodes as **0x00**. α -equivalence is absorbed by de Bruijn indices, assigned by position and not by name. $*@P \equiv P$ and $@*x \equiv_N x$ are absorbed by two rewrites applied during normalisation, confluent and terminating since each strictly decreases the count of $*@$ and $@*$ pairs. ($\Rightarrow$) By induction on the length of the encoding: the tag determines the constructor and LEB128 lengths determine field boundaries, so decoding is a function; the decoded term is congruent to any term with that encoding by the first direction applied to the representative. $\square$

Requirement 6.10 (Idempotence and stability). **normalise** **MUST** be idempotent through **un-normalise**, and **MUST** be invariant under permutation of the components of a par in the tree. Both **MUST** be property tested (Section 12).

Obligation 6.11 (Free levels and ordering interact). Levels are assigned by first occurrence in a left-to-right traversal, and ordering reorders the traversal. The implementation **MUST** therefore assign levels *after* ordering, in a traversal of the ordered term, and **MUST** reach a fixed point: order with the previous round’s levels, reassign, repeat. Two rounds suffice for the kernel; that claim is recorded here as an obligation to be proved or replaced by a loop with a proved bound. The same subtlety exists in **models** and is worth checking there.

Obligation 6.12 (Agreement with the node’s normaliser and sorter). For every kernel term, the **Norm** produced here and the **Par** produced by **Compiler::source_to_adt** followed by **Par-SortMatcher** **MUST** induce the same channel identity: equal terms give equal **Blake2b-256** hashes under the node’s serialisation. Established by differential test over the corpus, not by construction, because the two encodings differ.

6.5 Hashing

```
pub struct Hash32(pub [u8; 32]);
pub trait Digest32 { fn digest(bytes: &[u8]) -> Hash32; }
pub struct Blake2b256;           // default, in-crate, ~200 lines, no C
```

Requirement 6.13 (Hash discipline, caller side). `kindling-norm` MUST compute the content hash of a term exactly once, from `[-]`, and MUST expose it so that a `CAMPF1R3` caller satisfies that store’s `Keyed` contract without re-serialising. The digest MUST be a type parameter with `Blake2b256` as default. The implementation MUST NOT use a C library or a crate outside the allowlist of Section 11.

7 Instantiating CAMPF1R3

`kindling-campf1r3` is the only crate that names both projects. It contains type definitions and trait implementations, and SHOULD contain no algorithm.

CAMPF1R3 parameter	Instantiated at
C (channel)	Name in normal form: a quoted Norm , a free name level, or an unforgeable identifier. <code>content_hash</code> is the memoised hash of its encoding.
A (datum)	Norm . The send’s argument list becomes the datum sequence, so <code>CAMPF1R3</code> ’s arity check is the send’s arity.
P (pattern)	Pattern : a Norm whose free variables and wildcards are the binding sites, with the binder count precomputed.
K (continuation)	Cont { body : Norm , binders : u16 }, hashed over the body encoding together with the binder count.
Matcher	<i>not supplied here</i> . The kernel matcher is a separate deliverable; this specification guarantees only that patterns and data are in normal form when it is called, which is what makes a linear-time structural match possible.

Remark 7.1 (What this buys the store). `CAMPF1R3` forbids itself from hashing payloads, and notes that `rspace++` today serialises and hashes every candidate on every produce and consume. With the normaliser, the hash is computed once when the term is built and carried with it, so the matching loop compares 32 bytes to reject and calls the matcher only on survivors.

7.1 The WebAssembly interface

`kindling-wasm` exposes each crate behind a cargo feature, at the byte level, so that neither the JS nor the Swift side owns a Rust object graph, and a module that only parses does not carry a normaliser.

```
// feature = "parse"
#[wasm_bindgen] pub fn parse_to_tree(src: &str, level: u8) -> Box<[u8]>; // tree wire form
#[wasm_bindgen] pub fn diagnostics(src: &str, level: u8) -> String;      // JSON
#[wasm_bindgen] pub fn print_tree(tree: &[u8], style: u8) -> String;
// feature = "norm"
#[wasm_bindgen] pub fn normalise(tree: &[u8]) -> Box<[u8]>;             // normal form
#[wasm_bindgen] pub fn hash_of(norm: &[u8]) -> Box<[u8]>;              // 32 bytes
#[wasm_bindgen] pub fn scene(norm: &[u8]) -> Box<[u8]>;                 // Section 10.4
```

Requirement 7.2 (No stub builds). A `wasm32` build MUST NOT substitute a reduced or empty implementation of any function in this specification. If a target cannot support a feature, the build MUST fail at compile time. The precedent to avoid is explicit: `rholang-parser`’s `wasm32` stub returns a successful parse of the empty process for every input.

Requirement 7.3 (Total functions). No public function in any crate MUST panic, abort, or call `unwrap` on input-derived data, for any input byte string, including invalid UTF-8 boundaries,

unbalanced delimiters, 4 GB of open parentheses, an ill-formed tree, a truncated encoding, and adversarially deep nesting. This **MUST** be established by fuzzing (Section 12) and **MUST** be a release gate. **unsafe** **MUST NOT** appear in any crate.

8 Errors and recovery

Requirement 8.1 (Diagnostics, not exceptions). Every failure **MUST** be a **Diag** in a list, with a **Span**, a stable **code**, a severity, and an optional primary suggestion. Parsing **MUST** continue after a syntax error and **MUST** report every independent error in one pass, because an editor that shows one error at a time teaches the programmer to compile repeatedly. Normalisation **MUST** likewise accumulate binding errors rather than stopping at the first.

```
pub struct Diag { pub code: Code, pub span: Span, pub sev: Sev, pub note: Option<Note> }
pub enum Sev { Error, Warning }
```

Diag, **Code** and **Span** live in **kindling-ast**, so that a consumer can handle diagnostics from both crates uniformly. The code set is closed, and each group belongs to exactly one crate:

Group	Raised by	Codes
Lexical	-parse	UnknownByte, UnterminatedComment, NonAsciiIdent, SourceTooLarge
Syntax	-parse	Expected, Unbalanced, EmptySend, MissingBody, TooDeep
Level	-parse	NotInKernel, NeedsK1, NeedsK2, ArityAtK0
Well-formedness	-ast	BadId, Unreachable, DegeneratePar, SpanOutOfRange
Binding	-norm	DuplicatePatternVar, WildcardNotBindable, UnboundRequired, NewNameInPattern
Encoding	-norm	DecodeTruncated, DecodeBadTag, DecodeBadIndex

Requirement 8.2 (Recovery is structural). Recovery **MUST** be by synchronisation on the kernel’s delimiters — **}**, **)**, **|** at the current nesting depth — and the damaged region **MUST** be recorded as a **ProcNode::Error** covering its span, so the tree remains a tree over the whole source. A tree containing **Error** nodes **MUST** still print and **MUST** still support editor queries; it is not well-formed (Definition 4.6), so **normalise** **MUST** refuse it and return the diagnostics that produced it.

Requirement 8.3 (Level diagnostics name the level). A construct rejected only because of the accepted level **MUST** report **NeedsK1** or **NeedsK2** and **MUST NOT** report a syntax error. The editor uses this to offer “this term needs the store kernel” rather than “unexpected token”.

9 Editor services

These exist because **RHOBOTS** needs them, and are specified here rather than in the **visionOS** client so that a text editor, an LSP server and the headset share one implementation. They are split by crate on the same principle as everything else: anything about the source is in **-parse**, anything about the tree is in **-ast**, anything that needs identity is in **-norm**.

9.1 Stable identity

Requirement 9.1 (Node identity survives edits). **kindling-ast** **MUST** expose **NodeIds** stable across a reparse of an unchanged subtree, so that a robot in the scene keeps its position, animation state and physical continuity when the programmer edits a sibling. Identity **MUST** be computed as a function of the path from the root together with the hash of the subtree’s normal form, and **MUST NOT** be a counter over allocation order.

Remark 9.2 (The one place the split costs something). That definition needs the normaliser, which `-ast` may not depend on (Requirement 3.1). The resolution: `-ast` defines `NodeId` and an `Identify` trait with a path-only default; `-norm` provides the hash-backed implementation. An editor that links only the parser gets identity stable under sibling edits but not under a renaming; one that links both gets the full guarantee. RHOBOTS links both.

9.2 Subterm reparse and splice

```
// kindling-parse
pub fn parse_fragment<'s>(src: &'s str, opts: &Options) -> Parsed<'s>;
// kindling-ast
pub fn splice(tree: &Ast<'_>, at: ProcId, frag: &Ast<'_>) -> Ast<'static>;
pub fn subtree_source<'s>(tree: &Ast<'s>, at: ProcId) -> &'s str; // by span
```

Decision 9.3 (No incremental parsing in v1). Incremental reparse is the tree-sitter feature most obviously lost, and it is not missed. RHOBOTS edits structure, not text: a gesture builds a node, and `Builder` serves that directly. Where text is typed, it is typed into one subterm, and `parse_fragment` plus `splice` reparses that subterm only. Whole-file incremental parsing is reconsidered if the LSP server, not the headset, asks for it.

9.3 Printing

```
// kindling-ast
pub fn print(tree: &Ast<'_>, style: &PrintStyle) -> String; // Faithful | Plain
// kindling-norm
pub fn print_norm(t: &Norm, style: &PrintStyle) -> String; // via unnormalise
```

Requirement 9.4 (Round trip). For every well-formed tree a , `parse(print(a, Plain))` MUST yield a tree with the same normal form. For every source s that parses cleanly, `print(parse(s).tree, Faithful)` MUST reproduce s byte for byte, including comments and layout. For every normal form t , `normalise(unnormalise(t))` MUST equal t . All three MUST be property tested.

Remark 9.5 (Why two printing styles and not three). `Faithful` prints from the tree with its trivia table, and is what an editor writes back to a file when only part of the term was touched. `Plain` prints the tree in the crate’s own layout, discarding trivia, and is what the headset exports. A printed *normal form* is just `Plain` applied to `unnormalise`, so there is no third style: the stability that used to be called “canonical printing” is a property of the normal form, not of the printer.

9.4 Scene queries

```
// kindling-norm
pub fn top_level(t: &Norm) -> &[Norm]; // what can act now: the z=0 plane
pub fn continuation_of(t: &Norm) -> Option<&Norm>; // what sits behind a receive
pub fn free_names(t: &Norm) -> &[FreeName];
pub fn metrics(t: &Norm) -> Metrics; // size, depth, width per node
pub fn redex_candidates(t: &Norm) -> Vec<(u32, u32)>;
```

Requirement 9.6 (Candidates are syntactic). `redex_candidates` MUST report pairs of a top-level send and a top-level receive whose channels are equal as normal-form names — a hash comparison. It MUST NOT perform pattern matching, decide a race, or reduce. RHOBOTS uses it to decide which robots walk toward each other; the store decides what actually fires.

Remark 9.7 (Level of detail). `metrics` exists so the renderer can decide when a shrunk robot falls below legibility and is replaced by the elision glyph the RHOBOTS spec calls for. Size and depth are computed during normalisation at no extra traversal.

10 Performance

10.1 Complexity

Let n be source bytes, m the node count, w the width of the widest par, d the depth, s the mean encoded subterm size.

Crate	Stage	Time	Space
-parse	lex	$O(n)$, one pass	reused token buffer
-parse	parse	$O(n)$	$O(m)$ arena + $O(d)$ work stack
-norm	resolve	$O(m)$	$O(d)$ scope stack
-norm	order + encode	$O(m + \sum w \log w \cdot s)$	$O(m)$ shared encodings
-norm	hash	$O(\ [-]\)$	$O(1)$
-norm	substitute	$O(m)$	$O(m)$
-ast	print	$O(m + \text{output})$	$O(d)$

10.2 Targets

Measured on the reference platform (**aarch64** macOS, one thread) over a corpus of kernel terms derived from **rholang-parser/tests/corpus** plus generated terms.

Quantity	Target
Parse throughput	≥ 40 MiB/s native; ≥ 15 MiB/s under wasm32
Normalise + hash	≥ 100 MiB/s of encoding
Small term latency	$\leq 2 \mu\text{s}$ from text to hashed normal form for a term under 200 bytes; this is the RHOBOTS keystroke budget and the CAMPF1R3 deploy budget
Allocation	$O(1)$ arena growth events per parse; zero allocation per token; peak heap $\leq 8n$
Code size, -parse + -ast	≤ 80 KiB uncompressed in a wasm32 module, against 2.75 MB of generated C in the component it replaces
Code size, -norm	≤ 60 KiB uncompressed, so that a parse-only module is genuinely smaller than a full one and the split earns its keep
Cold start	≤ 1 ms to first parse in a fresh WebAssembly instance; no lazy table construction, no global initialisation

Requirement 10.1 (Benchmarks are a gate). A benchmark suite **MUST** run in CI on the reference platform, per crate, and **MUST** fail the build on a regression of more than 10% against the recorded baseline. Code size **MUST** be measured the same way, for each of the two WebAssembly feature sets separately. Targets above are stated relative to measurements, and the first task of phase 1 is to record the baseline, including the current tree-sitter parser’s throughput on the same corpus.

11 Portability, determinism and dependencies

Target	Level	Notes
x86_64/aarch64 Linux, macOS	MUST	Reference platforms.
wasm32-unknown-unknown	MUST	Browser and CAMPF1R3 demo; no stubs (Requirement 7.2).
aarch64-apple-visionos (and simulator)	MUST	RHOBOTS, through a C ABI or a Swift package wrapping the static library.
wasm32-wasip1	SHOULD	Non-browser hosts.
no_std + alloc	SHOULD	Achievable for all three core crates; the only <code>std</code> uses are <code>String</code> in printing and <code>Vec</code> .

Requirement 11.1 (Dependency allowlist). `kindling-ast`, `kindling-parse` and `kindling-norm` MUST build with `core` and `alloc` and, at most, `smallvec`, plus the one intra-family dependency each is permitted in Section 3. They MUST NOT depend, directly or transitively, on `tree-sitter` or any C library, `serde`, `bincode`, `prost`, `regex`, `getrandom`, `tokio`, `rayon`, `lazy_static`, or any crate of the `f1r3node-rust` or `rholang-rs` workspaces. `kindling-wasm` MAY add `wasm-bindgen`. An optional `serde` feature MAY derive encodings for `Diag` and `Metrics` only, never for `Norm`, whose encoding is normative.

Requirement 11.2 (Determinism). For one input and one `Options`, the tree, the normal form, its encoding, the content hash, the diagnostic list in order, and both printings MUST be identical bit for bit across targets, pointer widths, optimisation levels and toolchain versions within the supported MSRV range. No output MUST depend on hash-map iteration order, address values, floating point, locale, or time. Hash containers MUST NOT appear on any path affecting output order; where a map is needed it MUST be a sorted structure or keyed by a deterministic index.

Remark 11.3. Requirement 11.2 is the normaliser’s half of CAMPF1R3’s determinism profiles. A **Deterministic** store fed by a non-deterministic front end is not deterministic, and replay would diverge at the first term whose components were ordered differently on two machines.

Requirement 11.4 (MSRV, versioning and the wire forms). Each crate MUST declare an MSRV and hold it for the life of a minor version; MUST be `#![forbid(unsafe_code)]`; and MUST version independently. Two byte formats are part of the public interface and MUST each carry a one-byte format tag, changed only on a major version: the normal-form encoding of Section 6, and the tree wire form used by `kindling-wasm`. A change to the encoding changes every content hash, and therefore every channel identity in every store built with it; it is the most expensive change this family can make.

12 Validation

Kind	Crate	Content
Golden corpus	all	Every kernel term in <code>rholang-parser/tests/corpus</code> plus a kernel-specific corpus; snapshots of tree, normal form, encoding, hash and both printings.
Round trip	<code>-ast</code> , <code>-parse</code> , <code>-norm</code>	The three properties of Requirement 9.4, over generated terms and over the corpus.
Congruence	<code>-norm</code>	Generate P , apply random congruence-preserving rewrites (commute a par, α -rename, insert and remove Nil, wrap and unwrap <code>*@</code>), assert equal encodings; generate distinct terms, assert distinct encodings. This tests Proposition 6.9 in both directions.
Builder parity	<code>-ast</code>	A tree built through <code>Builder</code> and the same tree parsed from its own <code>Plain</code> printing MUST normalise identically. This is the gate on Requirement 3.2, and the gate on RHOBOTS gesture editing.
Independence	all	A build of <code>-parse</code> with <code>-norm</code> absent, and of <code>-norm</code> with <code>-parse</code> absent, MUST both succeed and pass their own suites. Checked in CI, because a dependency creeps in by accident, never on purpose.
Differential, syntax	<code>-parse</code>	Parse the corpus with <code>rholang-parser</code> and with <code>K1NDL1NG</code> ; assert the trees agree on the kernel fragment and that every form <code>K1NDL1NG</code> rejects is either outside the kernel or rejected by both.
Differential, identity	<code>-par</code>	For each corpus term, compare the normaliser's hash with the <code>Blake2b-256</code> of the node's sorted <code>Par</code> ; this discharges Obligation 6.12.
Differential, store	<code>-campf1r3</code>	Drive <code>CAMPF1R3</code> with normal-form terms and with the <code>ExactBytes</code> reference instantiation over the same scripted operation sequence; assert identical event logs.
Fuzzing	<code>-parse</code> , <code>-norm</code>	<code>cargo-fuzz</code> targets for <code>parse</code> , <code>normalise</code> on an arbitrary tree, <code>Norm::decode</code> and <code>splice</code> ; the gate for Requirement 7.3 is 24 hours clean per release.
Cross-target	all	Run the golden corpus under <code>wasm32</code> in a host and compare byte for byte with native output; the gate for Requirement 11.2.

13 Open decisions

1. **Name.** `K1NDL1NG` is the working name for the family: kindling is what you feed a campfire. Crate names `kindling-ast`, `kindling-parse`, `kindling-norm` follow from it. Decide before the repository is created.
2. **One repository or three.** The three core crates can share a cargo workspace and a version cadence, or live separately. A workspace is recommended: it makes the independence test

of Section 12 a CI job rather than an act of faith, while the crates still publish and version separately.

3. **Whether `-ast` is its own crate.** Folding it into `-parse` would leave two crates rather than three, at the cost of making a gesture-built tree depend on a lexer — which is exactly the dependency Requirement 3.2 exists to prevent. Keeping it separate is recommended, and is cheap: it has no dependencies and almost no logic.
4. **Guards at K2.** Whether `where` clauses are in v1 at all. Recommended: parse them behind a feature, carry the guard as an unevaluated subterm through normalisation, and let the executive hand it to `rho-pure-eval`. No crate here evaluates anything.
5. **new at K1.** Pure rho does not need it and RHOBOTS excludes it; it is included because a store kernel without unforgeable names cannot express a private channel. If kernel terms are never deployed directly, `new` moves to K2 and the normaliser shrinks.
6. **Polyadic at K0.** K0 is monadic here, following *Rhobots in Space*. If the renderer would rather draw a robot with n hands from the start, K0 becomes polyadic and one level’s worth of checking disappears.
7. **The kernel matcher.** A spatial matcher over normal-form patterns is the remaining piece between this family and CAMPF1R3, and the piece that makes a headset-local executive possible. It is out of scope here. Whether it is `kindling-match` or a CAMPF1R3 companion is a question about which repository owns pattern semantics; its input invariants are already fixed by Section 6.
8. **The interpreter.** Likewise unspecified, and likewise the executive cannot be built without it. It needs only `-norm` and a store, and Decision 1.5 is what keeps it from reimplementing de Bruijn arithmetic.

14 Delivery

Six phases, each ending in something runnable.

1. **Baseline and tree.** Workspace, CI, benchmark harness, recorded baseline measurements of `rholang-parser` on the shared corpus. `kindling-ast` complete: types, arena, builder, well-formedness check, both printings.
2. **K0 parser.** `kindling-parse` at K0: lexer, parser, diagnostics, recovery, fragment and splice; faithful round trip. At the end of this phase an editor can load, show and write pure rho.
3. **K0 normaliser.** `kindling-norm` at K0: binders, ordering, encoding, `Blake2b256`, `unnormalise`, substitution; congruence and builder-parity property tests. At the end of this phase name equality is a hash comparison and RHOBOTS has everything it needs from this family.
4. **WebAssembly.** `kindling-wasm` with the two feature sets, cross-target determinism tests, per-feature code-size gates.
5. **K1 and the store.** Joins, persistence, peek, polyadic, patterns, `new`, sequential-receipt sugar, in both crates; then `kindling-campf1r3` and the store differential test. At the end of this phase CAMPF1R3 in a browser runs on real terms instead of `ExactBytes`.
6. **Node adapter and K2.** `kindling-par` in the `f1r3node-rust` workspace, the identity differential test (Obligation 6.12), the node-compatible minter; then `where` guards behind the feature, if Section 13(4) says so.

Facts about existing code in this document were read from `F1R3FLY-io/f1r3node-rust` at branch `feature/cost-accounted-rho` and `F1R3FLY-io/rholang-rs` at `master`, on 17 September 2026.